Guía rápida del entorno (S0 → S1)

> Generada desde `01-Sesiones/S0-Presentacion-y-Entorno/slides/instalacion/contenido.py` (la misma fuente que las diapositivas `slides/instalacion/index.html`). No la edites a mano: cambia `contenido.py` y ejecuta `python generar.py`.

Para qué sirve: dejar tu equipo listo para S1, en el orden correcto, con comandos para copiar y pegar y una comprobación al final de cada paso. **Estado (09/11/2026):** los comandos de Python, conda, pytest, SQLite y MLflow están ejecutados en Linux en un entorno recién creado; lo de Windows (winget, Anaconda Prompt, VS Code) está **sin probar** y se indica en cada paso (ver el final).

Las tres reglas

1. **Sigue el orden** (tabla de abajo) y **no pases al siguiente paso** si lo que ves no coincide con «Debe salir».
2. **Cierra y abre la terminal** tras instalar un programa; si no, no lo reconoce.
3. **Mira el prompt**: para todo lo de Python debe empezar por `(env-ia-mlops)`.

El orden

Bloque	Qué	Terminal
A · Programas	Git · VS Code · Miniconda	CMD o PowerShell
B · Conda	solo conda-forge	Anaconda Prompt
C · Repositorio	clonar la rama develop	CMD o PowerShell
D · Entorno	crear · activar · humo	Anaconda Prompt
E · Pruebas para S1	VS Code · pytest · SQLite · MLflow · dato	Anaconda Prompt + VS Code
F · Nube	no bloquea S1	según el paso

Qué terminal usar

Terminal	Para
CMD o **PowerShell**	winget, Git, VS Code, clonar (bloques A y C)
Anaconda Prompt (Miniconda3)	conda y Python (B, D y E)
Terminal de **VS Code**	lo mismo que Anaconda Prompt, si el entorno está activo

Requisitos: Windows 10 (1809+) o 11; reserva unos 6 GB libres (estimación: el entorno ocupa unos 2,5 GB, medido en Linux, y hay que sumar la caché de descargas de conda y los programas); conexión estable; cuenta de GitHub; no hace falta ser administrador. Si tu equipo no te deja instalar programas,

usa la ruta sin instalación (Codespaces): sección «Ruta sin instalación: Codespaces» de `03-Entorno/guia-instalacion/index.html`.

¿Qué archivo de requisitos uso?

Archivo (`03-Entorno/conda/`)	Qué es	Quién lo usa	Comando
`environment.yml`	Receta de conda: Python 3.11 y **23 paquetes** con versión fijada (conda-forge)	**Tu equipo** (obligatorio)	`conda env create -f …`
`requirements-dev.txt`	Los mismos 23 paquetes, para pip	Codespaces, GitHub Actions; alternativa sin conda	`pip install -r …`
`requirements.txt`	Lo mínimo para ejecutar la API y el modelo (7 paquetes)	La imagen Docker de S2	`pip install -r …`

Las tres listas fijan las mismas versiones (una prueba automática lo vigila). No instales paquetes con `pip` dentro de `env-ia-mlops` salvo que se te pida: el verificador lo avisa («con otra version») y te da la línea que lo arregla: `conda install --force-reinstall paquete=versión`. `conda env update` **no** revierte lo que cambió pip (comprobado); sí repone un paquete que falte.

Paso a paso

A0 · Comprueba winget

Bloque A (Programas) · Terminal: CMD o PowerShell · **Validación:** Sin probar en Windows

winget es el instalador de Windows que usan los comandos de este bloque. Viene con Windows 10 (1809 o superior) y Windows 11.

```
winget --version
```

Debe salir: Un número de versión, por ejemplo `v1.x`.

Si falla:

- Si no se reconoce, instala «Instalador de aplicación» desde Microsoft Store, o usa el enlace de descarga manual de cada programa.
- Si tu equipo es corporativo y no te deja instalar programas, ve a «Si algo falla y cómo pedir ayuda» (al final): hay una ruta sin instalación (Codespaces).

A1 · Git y tu carpeta de trabajo

Bloque A (Programas) · Terminal: CMD o PowerShell · **Validación:** Parte probada en Linux · el resto sin probar en Windows

Git descarga el material del curso y guarda tus cambios. Cierra y vuelve a abrir la terminal después de instalar: si no, no lo reconoce.

```
winget install --id Git.Git -e --accept-source-agreements --accept-package-agreements
# en una terminal NUEVA
git --version
# cambia los datos
git config --global user.name "Tu Nombre"
# cambia los datos
git config --global user.email "tu@correo.com"
git config --global core.longpaths true
# una ruta corta, fuera de OneDrive y sin espacios
mkdir C:\cursos
```

Debe salir: `git version 2.x…` Los demás comandos no imprimen nada si salen bien.

```
git version 2.43.0
```

Salida real en Linux; en Windows dirá algo como `2.x.windows.1`.

Si falla:

- `git` no se reconoce: cierra y abre la terminal. Si sigue igual, descárgalo de git-scm.com/download/win.
- Las rutas largas (OneDrive, carpetas con espacios) dan errores en Windows: clona siempre en `C:\cursos`.

A2 · Visual Studio Code y sus extensiones

Bloque A (Programas) · **Terminal:** CMD o PowerShell · **Validación:** Sin probar en Windows

VS Code es el editor del curso: ejecuta scripts, cuadernos y terminales. Las 7 extensiones dan Python, Jupyter, contenedores y Codespaces.

```
winget install --id Microsoft.VisualStudioCode -e --accept-source-agreements --accept-package-agreements
# en una terminal NUEVA
code --version
code --install-extension ms-python.python
code --install-extension ms-python.vscode-pylance
code --install-extension ms-toolsai.jupyter
code --install-extension ms-azuretools.vscode-containers
code --install-extension ms-kubernetes-tools.vscode-kubernetes-tools
code --install-extension ms-vscode-remote.remote-containers
code --install-extension GitHub.codespaces
code --list-extensions
```

Debe salir: Un número de versión y un código; la lista final muestra las 7 extensiones.

Si falla:

- `code` no se reconoce: abre una terminal nueva o reinstala marcando «Agregar a PATH».
- Un identificador de extensión puede haber cambiado: búscala por nombre en el panel Extensiones (Ctrl+Shift+X).

A3 · Miniconda

Bloque A (Programas) · **Terminal:** Anaconda Prompt (Miniconda3) · **Validación:** Parte probada en Linux · el resto sin probar en Windows

Miniconda instala `conda`, que crea entornos de Python aislados con las mismas versiones para todo el grupo. Se usa desde «Anaconda Prompt», no desde CMD.

```
# en CMD o PowerShell
winget install --id Anaconda.Miniconda3 -e --accept-source-agreements --accept-package-
agreements
# ahora en «Anaconda Prompt (Miniconda3)», desde el menú Inicio
conda --version
conda env list
```

Debe salir: Una versión de conda y una lista con al menos el entorno `base`. **No instales nada en `base`.**

```
conda 26.9.4
```

Salida real en Linux; tu versión puede ser otra.

Si falla:

- `conda` no se reconoce: no estás en «Anaconda Prompt (Miniconda3)». Búscalo en el menú Inicio.
- Si winget no encuentra el paquete, descarga Miniconda desde anaconda.com/docs/getting-started/miniconda/main.

B1 · Conda solo con conda-forge

Bloque B (Conda) · Terminal: Anaconda Prompt (Miniconda3) · **Validación:** Probado en Linux

El Miniconda de Anaconda se niega a crear entornos hasta aceptar los términos de sus canales por defecto, aunque el entorno use solo conda-forge. Esta configuración deja conda-forge como único canal y evita el bloqueo.

```
conda config --add channels conda-forge
# si dice «CondaKeyError … not present», no pasa nada
conda config --remove channels defaults
conda config --set channel_priority strict
conda config --add default_channels https://conda.anaconda.org/conda-forge
conda config --show channels default_channels
```

Debe salir: Los cuatro primeros no imprimen nada. El último debe mostrar **solo conda-forge** en `default_channels`.

Si falla:

- Si no hiciste esto y conda dice `CondaToSNonInteractiveError`, es este paso.
- Si tu organización necesita licencia de Anaconda, consúltalo antes: otras opciones son aceptar los términos con `conda tos accept` o usar Miniforge.

Validación: Receta probada en Linux con conda 26.9.4 (VALIDACION.md); el aviso de términos de servicio solo lo da el Miniconda de Anaconda.

C1 · Clona el curso y abre la rama develop

Bloque C (Repositorio) · Terminal: CMD o PowerShell · **Validación:** Parte probada en Linux · el resto sin probar en Windows

Hasta que `develop` se fusione en `main`, el material está en `develop`. Un `git clone` a secas te deja en `main`, que está casi vacío (comprobado con `git ls-remote` el 09/11/2026).

```
cd C:\cursos
git clone --branch develop https://github.com/helicr/curso-ml-ia-desarrollo-produccion.git
cd curso-ml-ia-desarrollo-produccion
git branch --show-current
code .
```

Si ya clonaste y estás en main:

```
git fetch origin
git switch develop
```

Para traer novedades más adelante:

```
git pull
```

Debe salir: `git branch --show-current` imprime **develop** y VS Code abre la carpeta con `01-Sesiones`, `03-Entorno`, `04-Proyecto-Base`...

Si falla:

- Revisa tu conexión y que copiaste la URL entera. Si GitHub te pide iniciar sesión, hazlo con tu cuenta; si dice que no encuentra el repositorio, pide acceso al instructor.
- Si ves pocas carpetas, estás en `main`: usa la sección «Si ya clonaste».

Validación: `git clone --branch develop` probado contra GitHub el 09/11/2026 (clon superficial; el repositorio es público y su rama por defecto es main); `switch` probado con un repositorio de prueba. `code .`, en Windows, no.

D1 · Crea y activa el entorno env-ia-mlops

Bloque D (Entorno) · Terminal: Anaconda Prompt, en la carpeta del repositorio · **Validación:** Parte probada en Linux · el resto sin probar en Windows

`environment.yml` es la receta: Python 3.11 y 23 paquetes con versión fijada, solo de conda-forge. Descarga cientos de MB: lánzalo con buena conexión y no lo interrumpas.

```
cd C:\cursos\curso-ml-ia-desarrollo-produccion
conda env create -f 03-Entorno\conda\environment.yml
conda activate env-ia-mlops
```

Si el entorno ya existe: reponer lo que falte o aplicar cambios del archivo:

```
conda env update -f 03-Entorno\conda\environment.yml --prune
```

Rehacerlo desde cero:

```
conda deactivate
conda env remove -n env-ia-mlops -y
conda env create -f 03-Entorno\conda\environment.yml
```

Debe salir: Al final conda imprime cómo activarlo y el prompt cambia a **(env-ia-mlops)**. Si el prompt no lo muestra, el entorno NO está activo.

```
Preparing transaction: ...working... done
Verifying transaction: ...working... done
Executing transaction: ...working... done
#
# To activate this environment, use
#
#     $ conda activate env-ia-mlops
#
# To deactivate an active environment, use
#
#     $ conda deactivate

WARNING conda.conda_pypi.main:notify_conda_pypi_tip(158):
  Did you know? You can install many PyPI packages with conda
  using conda-pypi. Get started:
    https://bit.ly/4xXYt0B
```

Salida real en Linux (final del comando). Tardó 66 s con buena conexión; en Windows y con tu red puede tardar bastante más.

Si falla:

- `CondaToSNonInteractiveError`: haz antes el bloque B.
- `CondaValueError: prefix already exists`: el entorno ya existe; usa «Si el entorno ya existe» o «Rehacerlo desde cero» (más arriba en este paso).
- Se corta a mitad por la conexión o el disco: repite el comando; `conda clean --all` libera espacio.
- En PowerShell, `conda activate` necesita `conda init powershell` y una terminal nueva; es más fácil usar Anaconda Prompt.

Validación: Creación ejecutada en Linux desde environment.yml el 09/11/2026. En Windows solo se comprobó que resuelve (--dry-run).

D2 · Comprueba que Python es el del entorno

Bloque D (Entorno) · Terminal: Anaconda Prompt con (env-ia-mlops) activo, en la carpeta del repositorio · **Validación:** Probado en Linux

La prueba de humo importa las librerías del curso y recorre un mini ciclo completo: capas en SQLite, entrenamiento con MLflow y una API. No necesita Internet.

```
python --version
python -c "import sys; print(sys.executable)"
python 03-Entorno\conda\prueba_de_humo.py
```

Debe salir: Python 3.11.x, una ruta que contiene `env-ia-mlops` y, en la **última línea**, **HUMO OK**.

```
Python 3.11.18
<conda>/envs/env-ia-mlops/bin/python
python 3.11.18
  ok    pandas                  2.3.3
  ok    pyarrow                 26.0.1
…
  ok    papermill               2.7.0
  ok    jupytext                1.19.6
  ok    nbclient                0.11.0
  sqlite: tablas ['fs_cliente', 'master_ventas', 'ml_dataset', 'raw_ventas']
  mlflow: ejecucion 485900f4 | auc 0.701
  api: respuesta {'clase': 0}
  api: entrada invalida rechazada (422)
HUMO OK
```

Salida real en Linux (se acortó el centro). En Windows verás tus rutas.

Si falla:

- Si la ruta NO contiene el nombre del entorno, estás usando otro Python: activa el entorno y repite.
- Si sale **FALLA** junto a una librería, el entorno quedó incompleto: rehazlo (paso D1, «Rehacerlo desde cero»). Copia el mensaje completo para el instructor.

E1 · VS Code: mismo Python en scripts y cuadernos

Bloque E (Pruebas para S1) · **Terminal:** VS Code · **Validación:** Parte probada en Linux · el resto sin probar en Windows

El error más común es dejar el kernel equivocado en un cuaderno. Si el intérprete o el kernel no son los del entorno, los cuadernos no encuentran las librerías.

1. En la terminal con el entorno activo, abre la carpeta: `code .`
2. Ctrl+Shift+P → **Python: Select Interpreter** → elige **env-ia-mlops**.
3. Abre `03-Entorno/conda/prueba_notebook.ipynb` → arriba a la derecha **Seleccionar kernel** → **Python Environments** → **env-ia-mlops**.
4. Pulsa **Ejecutar todo**. La primera celda muestra una ruta con `env-ia-mlops`; la última imprime **NOTEBOOK OK**.

```
# solo si el kernel no aparece en la lista; reinicia VS Code después
python -m ipykernel install --user --name env-ia-mlops --display-name env-ia-mlops
```

Debe salir: Cada celda se ejecuta sin errores y la última imprime **NOTEBOOK OK**.

```
Installed kernelspec env-ia-mlops in ~/.local/share/jupyter/kernels/env-ia-mlops
```

Salida real del registro del kernel en Linux (tu ruta será otra).

Si falla:

- La primera celda falla con «El kernel NO es env-ia-mlops»: elige el kernel correcto (paso 3).
- No aparece el entorno en la lista: ejecuta el comando de este paso y reinicia VS Code.

Validación: El cuaderno se ejecutó con `jupyter nbconvert` en Linux (NOTEBOOK OK) y el comando del kernel se probó; los clics en VS Code no se probaron.

E2 · Pruebas del proyecto base

Bloque E (Pruebas para S1) · **Terminal:** Anaconda Prompt con (env-ia-mlops) activo, en la carpeta del repositorio · **Validación:** Probado en Linux

`04-Proyecto-Base` es el proyecto con el que se trabaja en S1. Sus pruebas confirman que el entorno hace lo que el curso espera. No hace falta instalar el paquete con `pip` para S1.

```
cd 04-Proyecto-Base
python -m pytest -q -m "not lento"
# opcional: incluye ejecutar los 4 cuadernos (más lento)
python -m pytest -q
```

Debe salir: Rápidas: **30 passed, 4 deselected**. Completas: **34 passed**. Cualquier `failed` o `error` hay que contarlo al instructor.

```
.............................                                        [100%]
30 passed, 4 deselected in 10.73s
.................................                                    [100%]
34 passed in 47.60s
```

Salida real en Linux. Los segundos dependen de tu equipo.

Si falla:

- Solo ves puntos y no la línea «30 passed»: tu copia es anterior al 09/11/2026. Actualiza con `git pull`.
- `ModuleNotFoundError`: entorno equivocado. Mira que el prompt empiece por (env-ia-mlops).
- Si falla alguna prueba, copia la salida completa y envíala; no la edites.

Validación: Ejecutado en un entorno recién creado desde environment.yml, sin instalar nada más.

E3 · SQLite

Bloque E (Pruebas para S1) · Terminal: Anaconda Prompt con (env-ia-mlops) activo, en la carpeta del repositorio · **Validación:** Parte probada en Linux · el resto sin probar en Windows

Todas las capas del curso (raw, master, fs, ml) viven en una base SQLite. Python ya trae SQLite; solo hay que comprobar la versión y que ejecuta consultas.

```
python -c "import sqlite3; print(sqlite3.sqlite_version,
sqlite3.connect(':memory:').execute('SELECT 1 AS entorno_listo').fetchone())"
```

Opcional: un visor para abrir las bases (DB Browser for SQLite):

```
winget install --id DBBrowserForSQLite.DBBrowserForSQLite -e --accept-source-agreements --
accept-package-agreements
```

Ábrelo, crea una base nueva y, en «Ejecutar SQL», ejecuta `SELECT 1 AS entorno_listo;`: una fila con el valor 1.

Debe salir: Una versión de SQLite (3.25 o superior) y `(1,)`.

```
3.54.1 (1,)
```

Salida real en Linux; tu versión de SQLite puede ser otra.

Si falla:

- `ModuleNotFoundError` o un error raro: el Python no es el del entorno. Repite la comprobación del paso D2.

Validación: El comando se ejecutó en Linux; el visor DB Browser, no.

E4 · MLflow: abre la interfaz

Bloque E (Pruebas para S1) · Terminal: Anaconda Prompt con (env-ia-mlops) activo, en la carpeta del repositorio · **Validación:** Parte probada en Linux · el resto sin probar en Windows

MLflow guarda parámetros, métricas y modelos de cada entrenamiento. En S1 lo usarás con una base SQLite. Aquí solo compruebas que arranca y se ve en el navegador.

```
# fuera del repositorio: crea un archivo mlflow.db que no queremos en el curso
cd C:\cursos
mlflow ui --backend-store-uri sqlite:///mlflow.db
```

1. Abre `http://127.0.0.1:5000` en el navegador: debe verse la pantalla de **Experiments**.
2. Para parar el servidor: **Ctrl+C** en esa terminal. Borra después `C:\cursos\mlflow.db`.

En S1 (desde 04-Proyecto-Base, después de ejecutar el cuaderno 04):

```
mlflow ui --backend-store-uri sqlite:///models/mlflow.db
```

Debe salir: La terminal se queda ocupada (es normal) mostrando `Uvicorn running on http://127.0.0.1:5000`. Tarda unos segundos en estar listo.

```
INFO:     Uvicorn running on http://127.0.0.1:5000 (Press CTRL+C to quit)
```

Salida real en Linux: tardó 12 s en responder.

Si falla:

- El puerto 5000 está ocupado: usa `mlflow ui --port 5001 --backend-store-uri sqlite:///mlflow.db` y abre el puerto 5001.
- El navegador no abre la página: espera 20 s y refresca; comprueba que escribiste `127.0.0.1`, no otra dirección.

Validación: El servidor se arrancó y respondió en Linux; el navegador en Windows, no.

E5 · El dato del caso: Bank Marketing

Bloque E (Pruebas para S1) · Terminal: Anaconda Prompt con (env-ia-mlops) activo, en la carpeta del repositorio · **Validación:** Parte probada en Linux · el resto sin probar en Windows

El caso que vas a desarrollar en el curso usa este archivo. Es de solo lectura: léelo con Python, no lo abras y guardes con Excel (cambiaría su contenido).

```
python -c "import pandas as pd; d = pd.read_csv('02-Proyecto-Transversal/datos/bank-marketing/bank-additional-full.csv', sep=';'); print(d.shape);
print(d['y'].value_counts().to_dict())"
```

Opcional: comprobar la huella a mano (PowerShell):

```
Get-FileHash -Algorithm SHA256 02-Proyecto-Transversal\datos\bank-marketing\bank-additional-full.csv
```

Debe coincidir con la línea de `02-Proyecto-Transversal/datos/SHA256SUMS`. El verificador del paso siguiente lo hace por ti.

Debe salir: `(41188, 21)`: 41 188 filas y 21 columnas (el separador es `;`), y el reparto de la variable objetivo `y`.

```
(41188, 21)
{'no': 36548, 'yes': 4640}
```

Salida real en Linux.

Si falla:

- Otras dimensiones: el archivo está modificado. Recupéralo con `git restore 02-Proyecto-Transversal/datos/bank-marketing/bank-additional-full.csv`.
- `FileNotFoundError`: no estás en la carpeta del repositorio, o clonaste `main` en lugar de `develop`.

Validación: El comando de Python se ejecutó en Linux; `Get-FileHash` es de Windows y no se probó.

E6 · La comprobación final: verificar_s1.py

Bloque E (Pruebas para S1) · Terminal: Anaconda Prompt con (env-ia-mlops) activo, en la carpeta del repositorio · **Validación:** Probado en Linux

Reúne las comprobaciones anteriores y otras (versiones frente a environment.yml, Git, disco, puerto 5000, huella del dato) y dice qué hacer en cada fallo. Si lo ejecutas con `--informe`, guarda `verificacion-s1.txt` para enviarlo al instructor.

```
cd C:\cursos\curso-ml-ia-desarrollo-produccion
python 03-Entorno\conda\verificar_s1.py --con-pruebas --informe
```

Debe salir: Todas las líneas en **OK** y la última: **LISTO PARA S1**. **AVISO** = revísalo, no bloquea. **FALLA** = arréglalo antes de S1 (la propia línea dice cómo).

```
Verificacion para S1 - 2026-11-09 20:52 - Linux 6.18.44-fc-v310 - Python 3.11.18
Entorno: env-ia-mlops   Repositorio: curso-ml-ia-desarrollo-produccion

  [OK   ] python                    3.11.18
  [OK   ] entorno                   env-ia-mlops
  [OK   ] paquetes                  23 paquetes con la version fijada en environment.yml
  [OK   ] importaciones             12 librerias clave se importan (2.8 s)
  [OK   ] sqlite                    version 3.54.1; consultas con funciones de ventana
correctas
  [OK   ] mlflow                    version 3.18.0; guarda y lee una ejecucion en SQLite
  [OK   ] kernel                    kernel «env-ia-mlops» registrado
  [OK   ] git                       git version 2.43.0
  [OK   ] git: identidad            nombre y correo configurados (no se muestran)
  [OK   ] repositorio               material del curso completo en la carpeta clonada
  [OK   ] dato Bank Marketing       huella correcta; 41188 filas de datos (+ encabezado)
  [OK   ] disco                     24 GB libres
  [OK   ] puerto 5000               libre para «mlflow ui»
  [OK   ] pruebas del proyecto base  30 passed, 4 deselected in 8.62s (11 s)

Resultado: LISTO PARA S1  (14 OK, 0 AVISO, 0 FALLA)

Informe guardado en verificacion-s1.txt (en la carpeta actual). Enviaselo al instructor si algo
no se resuelve.
```

Salida real en Linux. El informe no incluye tu nombre de usuario ni tu correo de Git.

Si falla:

- Sale **FALLA** en «python» y «entorno»: no tienes el entorno activo. Empieza por ahí: casi todo lo demás es consecuencia.
- Sale **AVISO** en «paquetes»: algún paquete tiene otra versión (suele ser un `pip install` posterior). La línea «que hacer» trae el comando exacto: `conda install --force-reinstall paquete=versión`. `conda env update` no lo arregla (comprobado).

Validación: Ejecutado en el entorno recién creado, con la identidad de Git puesta como en A1: termina en LISTO PARA S1. El verificador tiene pruebas propias en 03-Entorno/conda/pruebas.

F1 · Google Cloud y Codespaces (no bloquean S1)

Bloque F (Nube) · Terminal: CMD o PowerShell · **Validación:** Sin probar en Windows

Hacen falta desde S2 y S3. En S0 solo te autenticas y eliges tu proyecto: **no crees recursos de pago**. Docker y Kubernetes no se instalan en tu equipo: se usan dentro de Codespaces.

```
winget install --id Google.CloudSDK -e --accept-source-agreements --accept-package-agreements
# en una terminal NUEVA
gcloud --version
gcloud auth login
# el ID del proyecto, no su nombre
gcloud config set project TU_ID_DE_PROYECTO
gcloud auth list
```

Debe salir: Tu cuenta aparece con un asterisco (activa) en `gcloud auth list`.

Si falla:

- Si no puedes instalar gcloud, usa Cloud Shell desde el navegador: no necesita instalación ni autenticación local.
- Docker en Codespaces: sigue las diapositivas 23 y 25 de la presentación principal de S0.

Errores frecuentes

Lo que ves	Causa	Qué hacer
`conda` no se reconoce	Terminal equivocada.	Usa «Anaconda Prompt (Miniconda3)» desde el menú Inicio.
`CondaToSNonInteractiveError`	Términos de servicio de Anaconda sin aceptar.	Haz el bloque B (solo conda-forge) y repite la creación.
`CondaValueError: prefix already exists`	El entorno ya existe.	`conda env update -f 03-Entorno\conda\environment.yml --prune`, o bórralo y créalo.
`ModuleNotFoundError`	Entorno o kernel equivocado.	El prompt debe empezar por (env-ia-mlops). En un cuaderno, elige ese kernel.
Faltan carpetas del curso	Clonaste `main`.	`git fetch origin` y `git switch develop`.
pytest solo muestra puntos	Copia anterior al 09/11/2026.	`git pull`. Debe verse «30 passed».
«la huella no coincide»	El CSV de Bank Marketing se modificó.	`git restore` de ese archivo; no lo guardes con Excel.
`mlflow ui` no abre	Puerto 5000 ocupado o servidor aún arrancando.	`--port 5001`; espera 20 s y refresca.

Si algo falla y cómo pedir ayuda

Envía al instructor: (1) el comando que ejecutaste y en qué terminal; (2) el mensaje de error **completo**, como texto; (3) `verificacion-s1.txt` (lo crea `python 03-Entorno\conda\verificar_s1.py --informe`; no lleva tu usuario ni tu correo). **Nunca** envíes contraseñas, tokens ni claves.

Listo para S1 cuando...

- [ ] `git --version` responde y `git branch --show-current` dice **develop** *(A1 · C1)*
- [ ] `code --list-extensions` muestra las 7 extensiones *(A2)*
- [ ] `conda --version` responde en Anaconda Prompt *(A3)*
- [ ] `conda activate env-ia-mlops` y la prueba de humo termina en **HUMO OK** *(D1 · D2)*
- [ ] El cuaderno de prueba imprime **NOTEBOOK OK** con el kernel env-ia-mlops *(E1)*
- [ ] `pytest` del proyecto base: **30 passed** (y 34 con las lentas) *(E2)*
- [ ] SQLite, MLflow UI y el dato de Bank Marketing responden *(E3 · E4 · E5)*
- [ ] `verificar_s1.py` termina en **LISTO PARA S1** *(E6)*

Comandos de todos los días

Quiero...	Comando
Ver mis entornos (el activo lleva `*`)	`conda env list`
Activar / desactivar el entorno del curso	`conda activate env-ia-mlops` · `conda deactivate`
Ver qué hay instalado	`conda list`
Reponer un paquete que falta, o aplicar cambios del archivo	`conda env update -f 03-Entorno\conda\environment.yml --prune`
Devolver un paquete a la versión fijada (lo cambió pip)	`conda install --force-reinstall paquete=versión`
Saber en qué rama estoy / traer novedades	`git branch --show-current` · `git pull`
Pruebas rápidas del proyecto base	`python -m pytest -q -m "not lento"` (desde `04-Proyecto-Base`)
Pruebas cuyo nombre contiene «futuro» (S1)	`python -m pytest tests -k futuro -q`
Abrir MLflow en S1	`mlflow ui --backend-store-uri sqlite:///models/mlflow.db` (desde `04-Proyecto-Base`)
Comprobar todo otra vez	`python 03-Entorno\conda\verificar_s1.py`

Más detalle: `07-Recursos/guias/` (Miniconda, Git y ramas, VS Code, SQLite) y la guía interactiva `03-Entorno/guia-instalacion/index.html`.

Qué se probó y qué no

Probado el 09/11/2026 en Linux (kernel 6.18.44-fc-v310), en un entorno creado desde cero con `environment.yml`: creación del entorno, `HUMO OK`, versiones instaladas frente a las fijadas, 30 pruebas rápidas y 34 completas del proyecto base, SQLite, MLflow UI, lectura del CSV de Bank Marketing, registro del kernel, ejecución del cuaderno de prueba y `git clone --branch develop` contra GitHub (clon superficial). Las salidas que ves arriba son las reales de esa ejecución (`slides/instalacion/salidas-reales.json`).

Sin probar:

- Nada de esto se ejecutó en un Windows real: ni `winget`, ni «Anaconda Prompt», ni `code`, ni los clics en VS Code o en el navegador.
- El clonado se probó contra GitHub (clon superficial, Linux): trae la rama `develop` con las carpetas del curso. No se probó en Windows ni el clon completo de principio a fin con una conexión lenta.
- Los tiempos (creación del entorno, MLflow UI) son de Linux con buena conexión; en clase serán mayores.
- El aviso de términos de servicio de conda solo lo da el Miniconda de Anaconda; el conda usado aquí viene de conda-forge y no lo reproduce.